สรุปปลายภาค 88736065

API for Back-End Development · สรุปจาก wk09–wk14 และแนวข้อสอบปลายภาค

ข้อมูลสอบ

เช็กวันสอบก่อนอ่านต่อ

แนวข้อสอบระบุว่าสอบ **วันอังคารที่ 3 พฤศจิกายน 2569** และให้ตรวจวันเวลาตามประกาศคณะอีกครั้ง

พาร์ท 1 · ปรนัย 15 คะแนน

30 ข้อ ข้อละ 0.5 · wk09–14 สัปดาห์ละ 5 ข้อ · ไม่หักคะแนนข้อผิด · ห้ามเปิดเอกสาร · ทำใน Google Classroom · แนะนำเวลา 50 นาที

พาร์ท 2 · ปฏิบัติ 15 คะแนน

โปรเจกต์ระบบสั่งอาหารออนไลน์ Express + MySQL + Jest · 3 โจทย์ (6, 4, 5 คะแนน) · เปิดเอกสารบรรยายและแล็บ wk05–12 ได้ · แนะนำเวลา 60 นาที

- เวลาสอบ 180 นาที ข้อสอบออกแบบให้ทำจริงราว 2 ชั่วโมง ที่เหลือไว้ตรวจทาน
- ห้ามใช้ AI ช่วยตอบหรือเขียนโค้ด และห้ามคัดลอกโค้ดจากผู้อื่น ทั้งสองพาร์ท
- ปรนัยเน้นความเข้าใจเหตุผล ไม่ได้เน้นท่องนิยาม
- ส่งพาร์ท 2 ตามรูปแบบในข้อสอบ **ห้ามส่ง** `.env` และ `node_modules`
- เช็กเครื่อง: เปิด Docker Desktop, มี Node.js, ใน `final-exam-setup.zip` ต้องรัน `npm run check` เชื่อมฐานข้อมูลได้ และ `npm test` ผ่านครบ 3 เทสต์

เอกสารนี้ใช้ทบทวนก่อนสอบ

เอกสารที่เปิดได้ในพาร์ท 2 คือเอกสารบรรยายและแล็บ wk05–12 จึงควรเตรียมไฟล์เหล่านั้นไว้ และไม่นับเอกสารสรุปนี้เป็นเอกสารที่อนุญาต

สัปดาห์ 9: Unit & Integration Testing

Automated test รันซ้ำได้ ผลคงที่ และจับ regression ซึ่งการกด Postman ด้วยมือทำได้ไม่ดี

ระดับ	ทดสอบ	ความเร็ว/ปริมาณ	ตัวอย่าง
Unit	ฟังก์ชันเดียวแบบแยกอิสระ	เร็วสุด มีมากสุด	`hashPassword`
Integration	route + middleware + database ทำงานร่วมกัน	ปานกลาง	`POST /auth/login` คืน token
E2E	ทั้งระบบตั้งแต่ client ถึง DB จริง	ช้าสุด มีน้อยสุด	จำลองผู้ใช้กรอกฟอร์ม

Jest

- หาไฟล์ `*.test.js` หรือในโฟลเดอร์ `__tests__` อัตโนมัติ
- โครงสร้าง test: **Arrange** เตรียม → **Act** เรียกโค้ด → **Assert** ตรวจผล
- `toBe` เทียบแบบ `===` ใช้กับ primitive · `toEqual` เทียบค่าภายใน object/array
- `toThrow()` ใช้กับฟังก์ชัน sync: `expect(() => fn()).toThrow()` · Promise ใช้ `await expect(p).rejects.toThrow()`

- `beforeEach/afterEach` ทำทุก test ใช้รีเซ็ตสถานะ (test isolation) · `beforeAll/afterAll` ทำครั้งเดียว ใช้กับงานแพง เช่น เปิด/ปิด DB

Supertest

- ส่ง HTTP เข้า Express app ในหน่วยความจำ ไม่ต้องเปิดพอร์ตจริง จึงเร็วและไม่ชนพอร์ต
- ต้องแยก `app.js` (route, `module.exports = app`) ออกจาก `server.js` (`listen`) เพราะถ้า listen อยู่ไฟล์เดียวกัน พอ import มาเทสต์ เซิร์ฟเวอร์จะเริ่มทันที
- ทดสอบตั้งแต่ middleware ถึง handler จึงจัดเป็น Integration Test
- ชน DB จริงซ้ำ ๆ: ข้อมูลสะสม ชน UNIQUE ของ email รอบถัดไป · ต้องมี DB เปิดอยู่ · ช้า → แก้ด้วย **DB ทดสอบแยก + ล้างข้อมูล** หรือ **Mock**

Mocking

- `jest.fn()` สร้างฟังก์ชันจำลอง · `mockReturnValue()` สำหรับ sync · `mockResolvedValue()` สำหรับ async
- `jest.mock("./db")` แทนทั้งโมดูล ต้องอยู่ระดับบนสุดของไฟล์ เพราะ Jest ยก (hoist) ขึ้นไปก่อน `require`
- ควร mock: logic ของ route ที่ต้องการรันไม่พึ่ง DB, External API เช่น Payment · ไม่ต้อง mock: logic ล้วน · ไม่ควร mock: เมื่อทดสอบว่าต่อ DB จริงตรง schema

Coverage

มี 4 มิติ: Statement, Branch (ทุกทางของ if/switch), Function, Line คอลัมน์ `Uncovered Line #s` บอกบรรทัดที่ยังไม่มีเทสต์

> **จุดหลอก**
> Coverage 100% บอกแค่ว่าโค้ดถูกรัน ไม่ได้บอกว่าถูก เทสต์ที่ไม่มี `expect()` ก็เพิ่ม coverage ได้ แนวทางที่ถูกคือใช้ coverage หาจุดที่ขาด แล้วดูความสำคัญทางธุรกิจประกอบ ไม่ไล่ 100% ทุกไฟล์

สัปดาห์ 10: OpenAPI, Swagger & DX

DX คือประสบการณ์ของนักพัฒนาที่เรียกใช้ API เอกสารแย่ทำให้เกิด contract mismatch และเจ้าของ API กลายเป็นคอขวด

ส่วนของ OpenAPI	หน้าที่
`info`	ชื่อ คำอธิบาย เวอร์ชันของ API
`servers`	URL ของเซิร์ฟเวอร์ แยก dev/production ได้
`paths`	endpoint, method, parameter, requestBody, responses
`components.schemas`	data model ที่ใช้ซ้ำผ่าน `$ref`
`components.securitySchemes`	วิธียืนยันตัวตน เช่น JWT Bearer

- `$ref` แก้ schema จุดเดียวมีผลทุกจุด ลดเอกสารไม่ตรงกัน
- เอกสารต้องระบุ error ด้วย: 400 validation, 401 ไม่ยืนยันตัวตน, 403 ไม่มีสิทธิ์ RBAC, 404 ไม่พบ, 409 ข้อมูลซ้ำ
- เครื่องมือสร้างจาก OpenAPI ได้: หน้าเอกสาร (Swagger UI, Redoc), Client SDK, contract test, mock server (Prism)
- Swagger UI อ่าน `openapi.yaml` แล้วแสดงหน้าเว็บพร้อม Try it out ที่ `/api-docs` เอกสารอยู่ repo เดียวกับโค้ด จึงลด stale documentation ได้ **บางส่วน** ยังต้องอัปเดตตามโค้ดเอง
- Production ไม่ควรเปิด `/api-docs` สาธารณะ: เปิดเฉพาะ `NODE_ENV !== "production"` หรือใส่ auth middleware

Postman และ Newman

- Collection รวม request แชร์ได้ · ตัวแปร `{{baseUrl}}` สลับ dev/staging, `{{authToken}}` ใช้ token ซ้ำ
- เก็บ token อัตโนมัติ: `pm.environment.set("authToken", pm.response.json().token)`
- ควรมี Test script ทุก request และมี request ทดสอบกรณีผิดพลาดแยกด้วย
- ต้อง export ทั้ง `postman_collection.json` และ `environment.json` เพราะค่าตัวแปรอยู่ไฟล์หลัง
- Newman รัน Collection จาก command line: `newman run postman_collection.json -e environment.json` คล้าย Supertest แต่ทำงานที่ HTTP จริงแบบ black-box ไม่ผูกกับภาษา

API-First: ตกลง OpenAPI เป็นสัญญาก่อนเขียนโค้ด Frontend เริ่มคู่ขนานด้วย mock server ได้ ต่างจาก code-first ที่เขียนเอกสารตามหลัง

สัปดาห์ 11: External API & Resilience

เราคุมบริการภายนอกไม่ได้ เป้าหมายคือระบบต้องไม่ล่มตามเขา (graceful degradation)

Axios

- เทียบ fetch: มี option `timeout`, แปลง JSON ให้ใน `response.data`, throw เมื่อ status ไม่ใช่ 2xx, มี interceptor
- `error.response` มีค่า = เซิร์ฟเวอร์ตอบ 4xx/5xx · ไม่มีค่า = network error หรือ timeout · timeout มี `error.code === "ECONNABORTED"`

ควร retry ไหม

ควร (transient)	ไม่ควร (permanent)
Network error, Timeout, 5xx, 429 (พร้อม delay ยาวขึ้น)	400, 401, 404 เพราะลองซ้ำก็ผิดเหมือนเดิม

- **Fixed delay** รอเท่าเดิม ง่าย แต่ client retry พร้อมกันได้ · **Exponential backoff** 1s, 2s, 4s ลดภาระบริการที่กำลังฟื้น แต่ผู้ใช้รอนานขึ้น
- **Thundering herd:** client ที่ fail พร้อมกัน retry พร้อมกันเป็นคลื่น → แก้ด้วย **Jitter** สุ่มเวลาเพิ่ม
- **Full jitter** สุ่ม 0 ถึง base · **Equal jitter** base/2 + สุ่ม 0 ถึง base/2 (วิชานี้ใช้ มี delay ขั้นต่ำ)

```
const canRetry = attempt <= maxRetries && isRetryable(error);
const baseBackoffMs = 2 ** (attempt - 1) * 1000;
const backoffMs = baseBackoffMs / 2 + Math.random() * (baseBackoffMs / 2);
```

จุดหลอก: นับครั้งและเวลา

`maxRetries = 2` เรียกรวมสูงสุด **3 ครั้ง** เวลารวมที่ผู้ใช้อาจรอ = timeout × จำนวนครั้ง + ผลรวม backoff เช่น timeout 3 วินาทีกับ equal jitter: 9 + (1.5 ถึง 3) = **10.5 ถึง 12 วินาที**

Timeout และ Fallback

- ไม่มี timeout: ผู้ใช้รอไม่สิ้นสุด และ connection/memory ถูกใช้ค้าง กระทบผู้ใช้คนอื่น ทั้งที่ event loop ไม่ถูก block โดยตรงระหว่างรอ I/O
- ค่า timeout: ระหว่าง request ผู้ใช้ 2–5 วินาที · ดู SLA ของผู้ให้บริการ · งาน background ตั้งนานขึ้นได้
- Fallback: ค่า default, ค่าจาก cache เดิมพร้อมบอกว่าอาจไม่ล่าสุด, ตอบ 503 พร้อมข้อความชัดเจน, เลื่อนงานไปทำภายหลัง

Webhook

Outbound เราเรียกเขาเมื่อเราต้องการ · Webhook (inbound) เขาเรียกเราเมื่อเกิดเหตุการณ์ Endpoint ต้อง: **ตอบเร็ว** (200 ก่อน แล้วค่อยประมวลผลเบื้องหลัง), **ตรวจ signature**, **idempotent** (เช็ก event ID เพราะอาจส่งซ้ำ)

- HMAC: ผู้ส่งและผู้รับคำนวณจาก payload + secret แยกกัน ส่งเฉพาะ signature ไม่ส่ง secret ผ่านเครือข่าย ผู้ไม่รู้ secret ปลอมไม่ได้แม้รู้ว่าใช้ SHA-256
- ใช้ `express.raw()` ไม่ใช่ `express.json()` เพราะ HMAC ต้องคำนวณจาก raw bytes
- ใช้ `crypto.timingSafeEqual` แทน `===` กัน timing attack และต้องเช็กว่ามี header กับความยาว buffer เท่ากันก่อน มิฉะนั้นเกิด error จนได้ 500 แทน 401
- ไม่ผ่านตอบ **401** ทันที · ไม่ตรวจ signature = OWASP **A08** Software and Data Integrity Failures
- Signature อย่างเดียวไม่กัน replay attack ต้องดู timestamp ที่ถูกเซ็นร่วมกับ event ID

ผู้ให้บริการ	Header	เข้ารหัส	เซ็น timestamp
GitHub	`X-Hub-Signature-256`	hex	ไม่
Stripe	`Stripe-Signature`	hex	ใช่
Slack	`X-Slack-Signature`	hex	ใช่
LINE	`x-line-signature`	base64	ไม่
Shopify	`X-Shopify-Hmac-SHA256`	base64	ไม่

จุดหลอก

ใช้โค้ด hex กับ LINE/Shopify ความยาว buffer ไม่เท่ากัน ตรวจไม่ผ่านทุกครั้งแม้ secret ถูก ต้องเปลี่ยนเป็น base64 ทั้ง `Buffer.from` และ `.digest`

สัปดาห์ 12: Validation ขั้นสูงและ Rate Limiting

Validation ดู “รูปร่างข้อมูล” ส่วน Rate limiting ดู “จำนวนครั้ง” แทนกันไม่ได้ เพราะ request ที่ brute-force อาจมีรูปแบบถูกต้องครบ

ปัญหา	ตัวอย่าง	OWASP	แก้ด้วย
Mass assignment	ส่ง `role:"admin"` มาตอนสมัคร	A01	Zod
Type confusion	age เป็น `"25 OR 1=1"`	A05	Zod
Oversized payload	array หลายหมื่นแถว	A06	Zod `.max()` + limit ของ body
Brute-force	ยิง login รัว ๆ	A07	Rate limit

Zod

เครื่องมือ	ใช้เมื่อ	ตัวอย่าง
`z.object`	ตรวจแต่ละ field แยกกัน	`z.string().min(1)` , `z.email()`
`.refine()`	เทียบค่าข้าม field คืน true/false	endDate ต้องหลัง startDate

`.superRefine()`	กฎแบบ “ถ้า...ต้องมี...” เพิ่มได้หลาย issue ผ่าน `ctx.addIssue`	international ต้องมี passportNumber

- `.refine()` ทำงานหลังแต่ละ field ผ่านการตรวจเบื้องต้น และใช้ `path` ผูก error กับ field
- `safeParse` ไม่ throw คืน `{success, data}` หรือ `{success:false, error}` ส่วน `parse` throw
- Middleware `validate(schema)` ลดโค้ดซ้ำ ไม่ผ่านตอบ 400 `VALIDATION_ERROR` ผ่านแล้วตั้ง `req.body = result.data` และเรียก `next()`

Rate limiting

	Rate Limiting	Throttling
เกินโควตา	ปฏิเสธทันที **429**	หน่วงเวลา ยังตอบตามปกติ
เหมาะกับ	brute-force, DoS, โควตาแผนราคา	ลดภาระแบบค่อยเป็นค่อยไป
ไลบรารี	`express-rate-limit`	`express-slow-down`

- `windowMs` หน้าต่างเวลา · `max` โควตา · นับทุก request ไม่สนว่าผลเป็น 200 หรือ 401
- ต่าง endpoint ต่างโควตา เช่น login 5 ครั้ง/15 นาที เข้มกว่า general 100 ครั้ง/นาที
- **หลัง proxy:** ไม่ตั้งค่า `req.ip` เป็น IP ของ proxy ทุกคนถูกนับรวม → `app.set("trust proxy", 1)` และ proxy ต้องส่ง `X-Forwarded-For`
- **หลาย process:** counter อยู่ใน memory ของแต่ละ process เช่น PM2 4 process กับ `max:5` ผ่านได้ถึง 20 → ใช้ Redis ผ่าน `rate-limit-redis`
- **ตาม userId:** แม่นกว่า IP เมื่อผู้ใช้อยู่หลัง NAT เดียวกัน ต้องให้ `authenticateToken` ทำงานก่อน `userLimiter` และ fallback ใช้ `ipKeyGenerator(req.ip)` เพื่อ normalize IPv6

เครื่องมือทดสอบและ OWASP

- Postman Collection Runner: Iterations 20, Delay 0 จำลอง brute-force ดู 429 และ header `RateLimit`
- OWASP ZAP: proxy คั่นกลาง ใช้ Spider + Active Scan หา SQL Injection, XSS, header ที่ขาด แสดงระดับ High/Medium/Low/Informational แต่ไม่แทนการออกแบบความปลอดภัย
- มาตรการรวมกันเป็น defense in depth: A02 Helmet + ZAP · A05 parameterized query + Zod · A06 middleware + จำกัดขนาด · A07 JWT/RBAC + rate limit

สัปดาห์ 13: Microservices

Microservices ไม่ได้ดีกว่าเสมอไป ทีมเล็กควรเริ่มจาก Modular Monolith แล้วแยกเมื่อมีเหตุผลชัดเจน

ประเด็น	Monolithic	Microservices
ความซับซ้อนเริ่มต้น	ต่ำ เหมาะทีมเล็ก/MVP	สูง: เครือข่าย, service discovery, distributed data
Deploy	ทั้งระบบพร้อมกัน	แยกอิสระ
Scale	ทั้งระบบ	เฉพาะบริการที่ภาระสูง
ความล้มเหลว	อาจกระทบทั้งระบบ	ไม่ควรล่มทั้งระบบ ถ้าออกแบบ resilience
Consistency	ง่าย ใช้ transaction เดียว	ซับซ้อน ใช้ eventual consistency
ทดสอบ	end-to-end ง่ายกว่า	ต้อง mock หรือทดสอบข้ามบริการ

- **Bounded Context** (จาก DDD): แบ่งตามขอบเขตธุรกิจ เช่น Auth, Student, Enrollment, Notification
- **ไม่แบ่งตามชั้นเทคนิค** เช่น Database/API/Validation Service เพราะต้องเรียกกันตลอดจนเป็น **Distributed Monolith** ได้ข้อเสียทั้งสองแบบ
- เกณฑ์แบ่ง: ความรับผิดชอบธุรกิจ, อัตราการเปลี่ยนแปลง, ความต้องการ scale, ความเป็นเจ้าของข้อมูล, ทีมงาน

API Gateway

จุดเข้าถึงเดียวของ **client ภายนอก**: routing, auth กลาง, rate limit กลาง, logging/monitoring, รวม response จากหลายบริการ ถ้าไม่มี client ต้องรู้ URL ทุกบริการ และต้องเขียน auth/rate limit ซ้ำทุกบริการ

- ตัวอย่างในเอกสาร: Gateway ตัด `/api/v1` ก่อนส่งต่อ ใช้ Axios พร้อม timeout
- ปลายทางตอบ error (`error.response` มีค่า) → ส่ง status เดิมกลับ · ไม่ตอบเลย → **503** `SERVICE_UNAVAILABLE`

> **จุดหลอก**
> Service-to-service ในเอกสารเรียกตรงถึงกัน เช่น enrollment เรียก student ที่พอร์ต 4001 โดยไม่ผ่าน Gateway เพราะ Gateway เป็นทางเข้าของ client ภายนอก

การสื่อสารระหว่างบริการ

	Synchronous	Asynchronous
วิธี	HTTP/REST หรือ gRPC แล้วรอผล	ส่งเข้า Message Broker เช่น RabbitMQ, Kafka แล้วทำต่อทันที
เหมาะกับ	ต้องใช้ผลตัดสินใจขั้นต่อไป เช่น เช็กว่านิสิตมีจริง	ส่งอีเมล, log, แปลงวิดีโอ
ข้อเสีย	coupling ด้านเวลา ปลายทางล่ม/ช้ากระทบผู้เรียก	ต้องดูแล broker, eventual consistency, idempotency

Eventual consistency: ข้อมูลไม่ตรงกันทันที แต่จะตรงกันในที่สุด เกิดจาก database per service ที่ไม่มี transaction เดียวครอบทุกบริการ

Circuit Breaker: หยุดส่ง request ไปบริการที่ล้มเหลวซ้ำ ช่วย fail fast และไม่ซ้ำเติมบริการที่กำลังฟื้น ต่างจาก Retry ที่ยังลองต่อ

สถานะ	พฤติกรรม	เปลี่ยนเมื่อ
Closed	ปล่อย request ปกติ	ล้มเหลวเกินเกณฑ์ → Open
Open	ปฏิเสธทันที ไม่ส่งไปปลายทาง	ครบเวลา เช่น 30 วินาที → Half-Open
Half-Open	ส่ง request ทดลองจำนวนน้อย	สำเร็จ → Closed · ล้มเหลว → Open

สัปดาห์ 14: Migration และ Scalability

Migration บันทึกการเปลี่ยน schema เป็นเวอร์ชัน ทุกเครื่องได้ schema ตรงกันและมีแผนย้อนกลับ

Knex

```
npx knex migrate:make add_phone_number_to_students
npx knex migrate:latest      # รัน up ที่ยังไม่เคยรัน
npx knex migrate:rollback    # รัน down ของ batch ล่าสุด
```

```
exports.up = (knex) => knex.schema.alterTable("students", (table) => {
  table.string("phone_number", 20).nullable().defaultTo(null);
});
exports.down = (knex) => knex.schema.alterTable("students", (table) => {
  table.dropColumn("phone_number");
});
```

- ตาราง `knex_migrations` จำว่ารันไฟล์ไหนแล้ว จึงไม่รันซ้ำ
- ห้ามแก้ timestamp ในชื่อไฟล์ เพราะใช้เรียงลำดับ · ตั้งชื่อ `add_x_to_table` , `remove_x_from_table` , `create_x_table` · 1 ไฟล์ 1 การเปลี่ยนแปลง
- เลือก Knex เพราะใช้ `mysql2` เขียน query เดิมต่อได้ ไม่ต้องย้ายไป ORM อย่าง Sequelize/Prisma
- เพิ่ม column ในตารางที่มีข้อมูลต้อง `nullable()` หรือ `defaultTo()` ไม่อย่างนั้น NOT NULL ไร้ default จะล้มเหลว

จุดหลอก: down ของ backfill
ไม่มี down เลย `rollback` ใช้กับไฟล์นั้นไม่ได้ · backfill ย้อนค่าเดิมไม่ได้ ให้เขียน down เป็น `Promise.resolve()` พร้อม comment และแผนสำรองจริงคือ restore จาก backup

Expand-Contract: เปลี่ยนชื่อ column โดยไม่มี downtime

1. **Expand:** เพิ่ม column ใหม่ โดยยังไม่ลบของเดิม
2. **Migrate:** โค้ดเขียนลงทั้งสอง column, backfill ข้อมูลเก่า, แล้วย้ายการอ่านไปที่ column ใหม่
3. **Contract:** มั่นใจว่าไม่มีโค้ดอ้างถึงแล้ว จึงลบ column เดิม

แต่ละขั้นเป็น migration คนละไฟล์ และรองรับ rolling deployment ที่ instance เก่ากับใหม่รันพร้อมกัน

รันบน Production

Backup ด้วย `mysqldump` → ทดสอบบน staging ที่มีข้อมูลใกล้จริง → ตรวจ down → ใช้ Expand-Contract เมื่อกระทบ column ที่โค้ดใช้อยู่ → **migrate ก่อน แล้วค่อย deploy** → รันช่วงผู้ใช้น้อยและแจ้งแผนย้อนกลับ

Scaling

	Vertical (Scale Up)	Horizontal (Scale Out)
วิธี	เพิ่ม CPU/RAM/SSD เครื่องเดิม	เพิ่มจำนวน instance
ข้อดี	ง่าย ไม่แก้สถาปัตยกรรม	แทบไม่มีเพดาน ทนเครื่องล่ม rolling deploy ได้
ข้อเสีย	มีเพดาน มักมี downtime ตอนอัปเกรด ต้นทุนสูงขึ้น	ต้องมี Load Balancer, แอป stateless, DB เป็นจุดร่วม

- JWT ไม่เก็บ session ใน memory จึง stateless request ไปตก instance ไหนก็ได้
- ทีมเล็กเริ่ม Vertical ก่อน แล้ว Horizontal เมื่อชนเพดานหรือต้องการ high availability
- **Connection รวม = instance × connectionLimit** เช่น 3 × 10 = 30 ถ้าเกิน `max_connections` ได้ `ER_CON_COUNT_ERROR`
- ทางแก้: เพิ่ม `max_connections` (ใช้ RAM เพิ่ม ชั่วคราว), ลด `connectionLimit` (request อาจหน่วง), Read Replica (อ่านที่ replica เขียนที่ primary มี replication lag)

Load Balancer

อัลกอริทึม	หลักการ	เหมาะกับ
Round Robin	วนตามลำดับ	เครื่องสเปกเท่ากัน งานขนาดใกล้กัน

Least Connections	ส่งไปเครื่องที่ connection ค้างน้อยสุด	เวลาประมวลผลต่างกันมาก
IP Hash	hash IP ส่งเครื่องเดิมเสมอ	ยังต้องใช้ sticky session

Nginx: `upstream` รายการเครื่อง + `proxy_pass` และต้องมี `proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for` ไม่อย่างนั้น rate limit จะนับทุกคนเป็นคนเดียว (เชื่อมกับ wk12)

พาร์ท 2: ปฏิบัติ

ยังไม่ทราบโจทย์จริง
แนวข้อสอบบอกแค่ว่าเป็นโปรเจกต์สั่งอาหาร Express + MySQL + Jest มี 3 โจทย์ ตัวอย่างโดเมนด้านล่างจึงเป็นตัวอย่างฝึกที่แต่งเอง และเอกสาร wk05–08 ไม่อยู่ในไฟล์ที่ได้รับ ส่วนนี้จึงไม่ครอบคลุมเนื้อหาสัปดาห์เหล่านั้นโดยตรง ควรเปิดเอกสารจริงเทียบ

ลำดับทำงานในห้องสอบ

1. เปิด Docker และรัน `npm run check` กับ `npm test` เพื่อดูสภาพเริ่มต้น
2. อ่านโจทย์และข้อกำหนดว่าแก้ไฟล์ใดได้ ก่อนแตะโค้ด
3. เริ่มโจทย์ที่ทำได้เร็ว เก็บคะแนนก่อน โจทย์ 1 มี 6 คะแนน
4. อ่าน error ของ Jest ให้ครบ ดู test name, Expected/Received และบรรทัดที่ชี้ แล้วแก้ทีละจุด
5. ก่อนส่ง รัน `npm test` ซ้ำ ลบ `.env` และ `node_modules` ออกจากไฟล์ส่ง

โครงเทสต์ Supertest

```
const request = require("supertest");
const app = require("../app");

describe("POST /orders", () => {
  test("คืน 400 เมื่อไม่ส่ง items", async () => {
    // Arrange
    const payload = { customerName: "A" };
    // Act
    const res = await request(app).post("/orders").send(payload);
    // Assert
    expect(res.status).toBe(400);
    expect(res.body.error.code).toBe("VALIDATION_ERROR");
  });
});
```

แนบ token: `.set("Authorization", `Bearer ${token}`)`

Mock ฐานข้อมูล

```
jest.mock("../db");               // บนสุดของไฟล์
const pool = require("../db");

test("คืน 404 เมื่อไม่พบเมนู", async () => {
  pool.query.mockResolvedValueOnce([[]]);  // [rows] แบบไม่มีแถว
  const res = await request(app).get("/menus/999");
  expect(res.status).toBe(404);
});
```

ถ้าเทสต์ชน MySQL จริง ใช้ `beforeAll/afterAll` ล้างข้อมูลเฉพาะของเทสต์ และเรียก `await pool.end()` ตอนจบ

Route ที่ปลอดภัย

```
router.get("/orders/:id", authenticateToken, async (req, res) => {
  const [rows] = await pool.query(
    "SELECT * FROM orders WHERE id = ?",   // parameterized ห้ามต่อ string
    [req.params.id]
  );
  if (rows.length === 0) {
    return res.status(404).json({
      error: { code: "NOT_FOUND", message: "ไม่พบคำสั่งซื้อ" },
    });
  }
  return res.status(200).json({ data: rows[0] });
});
```

Zod สำหรับคำสั่งซื้อ

```
const createOrderSchema = z.object({
  items: z.array(z.object({
    menuId: z.number().int().positive(),
    quantity: z.number().int().min(1).max(20),
  })).min(1).max(50),
});
router.post("/orders", authenticateToken, validate(createOrderSchema), createOrder);
```

Status code ที่ต้องแม่น

Code	ใช้เมื่อ
200 / 201	สำเร็จ / สร้างใหม่สำเร็จ
400	ข้อมูลไม่ผ่าน validation
401	ไม่มี token, token ผิด, login ผิด, signature ผิด
403	ยืนยันตัวตนแล้ว แต่ไม่มีสิทธิ์ตาม role
404 / 409	ไม่พบข้อมูล / ข้อมูลซ้ำ

429	เกิน rate limit
503	บริการปลายทางไม่พร้อม

อาการที่พบบ่อยและจุดที่ควรเช็ก

รวมจากประสบการณ์ทั่วไปกับ Express และ Jest ไม่ได้มาจากเอกสารทั้งหมด

อาการ	เช็กอะไร
`ECONNREFUSED` ตอนต่อ DB	Docker เปิดไหม ค่าใน `.env` ตรงไหม
`req.body` เป็น undefined	มี `express.json()` ก่อน route ไหม
ได้ 404 ทั้งที่มี route	path/prefix ใน `app.use` และ method ตรงไหม
Expected 200 Received 500	ดู console: ชื่อตาราง/column ผิด ลืม `await` หรือลืม destructure `[rows]`
เทสต์ timeout หรือค้าง	middleware ลืม `next()`, handler ไม่ตอบ, ไม่ปิด pool
รันรอบแรกผ่าน รอบสองพัง	ข้อมูลซ้ำใน DB ต้องล้างข้อมูลกดสอบ
`Cannot find module`	path ใน `require` เทียบจากไฟล์เทสต์

ถ้าโจทย์ให้อธิบายสาเหตุ

แนวข้อสอบให้อธิบายด้วยประโยคของตัวเอง ใช้โครงนี้: “เทสต์คาดว่า X แต่ได้ Y เพราะโค้ดบรรทัด Z ทำ... จึงแก้โดย...”

จัดทำจากเอกสาร wk09–wk14 และแนวข้อสอบปลายภาค 88736065 จุดหลอกเป็นการเรียบเรียงเพื่อทบทวน